CS:5980
Foundations of Embedded Systems

Liveness Requirements
Part I

Copyright 2014-26, Rajeev Alur and Cesare Tinelli.
Created by Cesare Tinelli at the University of Iowa from notes originally developed by Rajeev Alur at the University of Pennsylvania. These notes are copyrighted materials and may not be used in other course settings outside of the University of Iowa in their current form or modified form without the express written permission of one of the copyright holders. During this course, students are prohibited from selling notes to or being paid for taking notes by any person or commercial firm without the express written permission of one of the copyright holders.

From Desktops to Cyber-Physical Systems

Traditional computers: Stand-alone devices running software applications

- e.g., data processing

Traditional controllers: Devices interacting with physical world via sensors and actuators

- e.g., thermostat

Embedded (aka Cyber-physical) Systems: Special-purpose system with integrated microcontroller/software

- e.g., cameras, watches, washing machines, …

Formal Verification



How to formalize requirements?

1. **Safety requirements:** Invariants, monitors
2. **Liveness requirements:** Temporal logic formulas

Recap: Safety Requirements

Nothing bad ever happens

- Trains should not be on bridge simultaneously
- If the east train is waiting, the west train should not be allowed on the bridge twice in succession

Violation of a safety property is provable by a (finite) execution prefix

Recap: Safety Requirements

Formalization:

- Identify a property φ over state variables, and check if φ is an invariant of the given system S, or
- Construct a monitor M and check that "monitor mode is not error" is an invariant of the composite system S || M

Analysis:

- Proof based on inductive invariants
- Algorithms for exploring the reachable states of the system

Liveness Requirements

Something good eventually happens

- A waiting train is eventually allowed to enter the bridge
- Each process eventually decides to be a leader / follower

Violation of a liveness property is not provable by a (finite) execution prefix

- but is provable by an execution where the system gets stuck in a cycle without achieving the goal

Liveness Requirements

Formalization:

- Need to consider infinite executions (ω-executions)
- Need a logic to state properties of such executions

Analysis:

- Proof typically based on automata constructions
- Special cases admit logical proofs

Temporal Logics

Logics proposed to reason about time

- Origins in philosophy
- Tense logic: Prior (1920)

Linear Temporal Logic (LTL) proposed to reason about executions of reactive systems

- Pnueli (1977), later selected for Turing award (1996)

Industrial adoption

- Property Specification Language (PSL) IEEE standard
- LTL enriched with many additional constructs for usability
- Supported by CAD tools for simulation/analysis of Verilog/VHDL

Valuations and Base Formulas

V: set of typed variables

- Example: nat x, bool y

Valuation: type-consistent assignment of values to variables in V

- q_0 : (x = 26, y = 0)
- q_1 : (x = 11, y = 1)

Base formula: Boolean-valued expression over V

- even(x)
- (y = 0) ⇒ even(x)

Satisfiability: valuation q satisfies formula φ, written q ⊨ φ, if q(φ) evaluates to 1

- q_0 ⊨ even(x)
- q_0 ⊨ (y = 0) ⇒ even(x)
- q_1 ⊭ even(x)
- q_1 ⊨ (y = 0) ⇒ even(x)

Traces

A base formula expresses a property of a single assignment

Trace: Infinite sequence of assignments

- ρ : (0,0), (1,1), (2,0), (3,1), (4,0), (5,1), …
- ρ' : (0,0), (21,1), (13,1), (43,0), …

In system specification and verification, V can be one of the following:

- The set of state variables
  (a trace is a possible infinite *execution* of the system)
- The set of input and output variables
  (a trace is an observed *input/output behavior* of system)
- The set of all state, input, and output variables

LTL Basics

A base formula expresses a property of single assignments

Trace: Infinite sequence of assignments

LTL formulas are built from Boolean-valued expressions using

- Logical connectives: $_ \wedge _$, $_ \vee _$, $_ \Rightarrow _$, $\neg _$
- Temporal operators: Always $_$, Eventually $_$, Next $_$, $_$ Until $_$
  $\Box _$, $\Diamond _$, $\circ _$, $_ \, U \, _$

Note: Base formula are just formulas with no temporal operators

LTL formulas are evaluated with respect to a trace

A trace $\rho = q_1, q_2, q_3, \ldots$ *satisfies* a base formula φ if $q_1 \models \varphi$ (i.e., assignment q_1 satisfies φ)

Always Operator

Always φ intuitively means φ holds at all times

For a base formula φ, a trace $\rho = q_1, q_2, q_3, \ldots$ *satisfies* Always φ if $q_j \models \varphi$ for all $j > 0$

Example: Trace

x:	0	1	2	3	4	5	…
y:	0	1	0	1	0	1	…

- falsifies (i.e., does not satisfy) Always even(x)
- satisfies Always (y = 0 ⇒ even(x))

Note: A state property φ is invariant for a transition system T iff every execution of T satisfies Always φ

Eventually Operator

Eventually φ intuitively means φ holds at some point (at least once)

For a base formula φ, a trace $\rho = q_1, q_2, q_3, \ldots$ *satisfies* Eventually φ if $q_j \models \varphi$ for some $j > 0$

Example: Trace

x:	0	1	2	3	4	5	…
y:	0	1	0	1	0	1	…

- satisfies Eventually (y = 1)
- satisfies Eventually (x = 45)
- falsifies Eventually (x = 4 ∧ y = 1)

Note: Eventually is the logical dual of Always:

a trace ρ satisfies Eventually φ iff ρ satisfies $\neg$Always $\neg\varphi$ iff

ρ falsifies Always $\neg\varphi$

Next Operator

Next φ intuitively means φ holds the *next* time

For a base formula φ, a trace $\rho = q_1, q_2, q_3, \ldots$ *satisfies* Next φ if $q_2 \models \varphi$

Example: Trace

x:	0	1	2	3	4	5	…
y:	0	1	0	1	0	1	…

- satisfies Next (y = 1)
- falsifies Next (x = 2)

Until Operator

φ Until ψ intuitively means ψ holds at some point and φ holds at all times until then

For base formulas φ, ψ, a trace $\rho = q_1, q_2, q_3, \ldots$ satisfies φ Until ψ
if $q_j \models \psi$ for some $j > 0$ and $q_i \models \varphi$ for all $i < j$

Example: Trace `x: 0 0 0 2 2 5 …`

- satisfies $(x = 0)$ Until $(x = 2)$
- satisfies $(x <= 2)$ Until $(x = 5)$

Note: If a trace satisfies φ Until ψ, it must also satisfy Eventually ψ

Nested Operators

Which traces satisfy Next Always φ where φ is a base formula?

- Trace $\rho = q_1, q_2, q_3, \ldots$ satisfies Next Always φ if $q_j \models \varphi$ for all $j > 1$

To formalize this, we need to define the relation $(\rho, j) \models \psi$ (ρ *satisfies* φ *at position* j) for arbitrary temporal formulas

- Same as *suffix* trace $q_j, q_{j+1}, q_{j+2}, \ldots$ (starting at position j) *satisfies* φ
- $(\rho, j) \models \varphi$ if $q_1 \models \varphi$ if φ is a base formula
- $(\rho, j) \models$ Next φ if $(\rho, j + 1) \models \varphi$
- $(\rho, j) \models$ Always φ if $(\rho, k) \models \varphi$ for all positions $k \geq j$
- $(\rho, j) \models$ Eventually φ if $(\rho, k) \models \varphi$ for some position $k \geq j$
- $(\rho, j) \models \varphi_1$ Until φ_2 if there is a position $k \geq j$ such that $(\rho, i) \models \varphi_1$ for all $i = j \ldots k - 1$ and $(\rho, k) \models \varphi_2$

Trace ρ *satisfies* temporal formula ψ iff $(\rho, 1) \models \psi$

Multiple Eventualities

Example: Multi-agent system where multiple goals have to be satisfied

- Goal1: Robot 1 has finished its mission
- Goal2: Robot 2 has finished its mission

Spec, version 1: (Eventually Goal1) $\wedge$ (Eventually Goal2)

- Trace ρ satisfies this spec if there are positions i, j such that $(\rho, i) \models$ Goal1 and $(\rho, j) \models$ Goal2
- No specific order in which goals are to be achieved

Spec, version 2: Eventually [Goal1 $\wedge$ (Eventually Goal2)]

- Trace ρ satisfies this spec if there are positions i, j such that $i \leq j$ and $(\rho, i) \models$ Goal1 and $(\rho, j) \models$ Goal2

Spec, version 3: Eventually [Goal1 $\wedge$ Next (Eventually Goal2)]

- Trace ρ satisfies this spec if there are positions i, j such that $i < j$ and $(\rho, i) \models$ Goal1 and $(\rho, j) \models$ Goal2

Recurrence and Persistence

Repeatedly φ = Always Eventually φ

- i.e., for all j, $(\rho, j) \models$ Eventually φ
- i.e., for all j, there is an $i \geq j$ such that $(\rho, i) \models \varphi$
- i.e., there are infinitely many suffixes of ρ that satisfy φ

Persistently φ = Eventually Always φ

- i.e., for some j, $(\rho, j) \models$ Always φ
- i.e., there is a j such that for all $i \geq j$, $(\rho, i) \models \varphi$
- i.e., formula φ is satisfied eventually and stays satisfied

The two patterns are logical duals:

- a trace ρ satisfies Repeatedly φ iff it falsifies Persistently $\neg\varphi$

Exercise

Trace:

x:	0	1	2	3	4	5	6	7 …
y:	0	1	0	1	0	1	0	1 …

Which of the following formulas are satisfied by the trace above?

1. Repeatedly (y = 0)
2. Persistently (x ≥ y)
3. Always [even(x) ⇒ Next odd(x)]
4. Always [even(y) ⇒ Next odd(x)]
5. Repeatedly prime(x)

Requirements-based Design

Given:

- Input/output interface of system C to be designed
- Model E of the environment
- LTL-formula φ over I/O variables and state variables of the environment model E

Design problem:

Fill in details of C so that every infinite execution of the composition of E and C satisfies φ

Applies to synchronous as well as asynchronous designs

Leader Election

Requirements refer to output variable $status_n$ of each node n

Liveness: Each node n eventually decides its status

$$\text{Eventually} (status_n = \text{leader} \vee status_n = \text{follower})$$

Safety: For distinct nodes m, n, if m decides to be a leader at any point at all, then n can never be a leader

$$\text{Eventually} (status_m = \text{leader}) \Rightarrow \text{Always} \neg(status_n = \text{leader})$$

Railroad Controller

Requirements refer to mode variables of trains and I/O variables (signals)

Safety: The two trains should not be on bridge simultaneously

$$\text{Always } \neg(\text{mode}_W = \text{bridge} \wedge \text{mode}_E = \text{bridge})$$

Liveness 1: West train gets on bridge repeatedly

$$\text{Repeatedly } (\text{mode}_W = \text{bridge})$$

Not a good spec (why?), no controller can satisfy this

Liveness 2: A waiting west train is eventually allowed to enter

$$\text{Always } [\ (\text{mode}_W = \text{wait}) \Rightarrow \text{Eventually } (\text{signal}_W = \text{green})\ ]$$

Note: LTL helps clarify ambiguities in English sentences. Liveness 2 is not satisfied by our controller (what is a counter-example?) What if east train never leaves the bridge?

Railroad Controller

Liveness 2': Conditioned upon east train not staying on bridge forever

$$\text{Repeatedly } \neg(\text{mode}_E = \text{bridge}) \Rightarrow (\text{Always}[\ (\text{mode}_W = \text{wait}) \Rightarrow \text{Eventually } (\text{signal}_W = \text{green})\ ])$$

Does either of the two controllers in Chapter 3 satisfy this?

Liveness 3: If west train is waiting, then eventually either it is allowed to enter bridge or east train is on bridge (implies absence of deadlocks)

$$\text{Always } [\ (\text{mode}_W = \text{wait}) \Rightarrow (\text{Eventually } (\text{signal}_W = \text{green} \ \vee \ \text{mode}_E = \text{bridge}\ )\ ])$$

Writing precise requirements is challenging but crucial!

Credits

Notes based on Chapter 5 of

Principles of Cyber-Physical Systems
by Rajeev Alur
MIT Press, 2015